저장소 구현체 비교 분석

개요

iroh-blob은 다양한 사용 사례와 환경에 맞춰 세 가지 주요 저장소 구현체를 제공합니다. 각 구현체는 서로 다른 트레이드오프와 최적화 전략을 가지고 있으며, 메모리 사용량, 성능, 영속성, 확장성 측면에서 차별화된 특성을 보입니다.

저장소 구현체 개요

1. MemStore (메모리 저장소)

- **위치**: `src/store/mem.rs`
- **특징**: 모든 데이터를 메모리에 저장하는 완전한 인메모리 저장소
- **용도**: 소규모 데이터, 테스트 환경, 임시 저장

2. FsStore (파일 시스템 저장소)

- **위치**: `src/store/fs.rs`
- **특징**: 하이브리드 접근법을 사용하는 영구 저장소
- **용도**: 대용량 데이터, 프로덕션 환경, 영구 저장

3. ReadonlyMemStore (읽기 전용 메모리 저장소)

- **위치**: `src/store/readonly_mem.rs`
- **특징**: 생성 시점에 제공된 데이터만 서비스하는 단순한 저장소
- **용도**: 정적 데이터 서비스, 테스트, 경량 구현

MemStore 상세 분석

구현 방식

```rust
#[derive(Debug, Clone)]
#[repr(transparent)]
pub struct MemStore {
client: ApiClient,
}

struct State {
data: HashMap<Hash, BaoFileHandle>,
tags: BTreeMap<Tag, HashAndFormat>,
empty_hash: BaoFileHandle,
}
```

핵심 특징

1. 완전한 메모리 기반 저장

- 모든 블롭 데이터가 메모리에 저장됨
- `HashMap<Hash, BaoFileHandle>` 구조로 해시별 데이터 관리
- 빠른 랜덤 액세스 제공

2. 동적 데이터 구조

```rust
#[derive(Debug, derive_more::From)]
pub  enum BaoFileStorage {
Partial(PartialMemStorage),
Complete(CompleteStorage),
}
```

3. 실시간 상태 전환

- 부분 다운로드에서 완전한 데이터로 동적 전환
- 스트리밍 중 실시간 검증 및 상태 업데이트

장점

1. **최고 성능**

- 메모리 액세스 속도로 인한 극도로 빠른 읽기/쓰기
- 디스크 I/O 오버헤드 없음
- 즉시 데이터 접근 가능

2. **단순한 구조**

- 복잡한 파일 시스템 관리 불필요
- 동시성 처리가 상대적으로 단순
- 메모리 할당/해제만 관리

3. **완전한 기능 지원**

- 태그 시스템 완전 지원
- 가비지 컬렉션 지원
- 부분 다운로드 및 스트리밍 지원

단점

1. **메모리 제약**

- 사용 가능한 메모리에 의해 저장 용량 제한
- 대용량 파일 처리 시 메모리 부족 위험
- 메모리 사용량이 데이터 크기에 비례

2. **데이터 영속성 부족**

- 프로세스 종료 시 모든 데이터 손실
- 시스템 재시작 후 데이터 복구 불가능
- 백업 및 복구 메커니즘 부재

3. **확장성 제한**

- 단일 머신의 메모리 용량에 제한
- 수평 확장 어려움

FsStore 상세 분석

하이브리드 아키텍처

FsStore는 데이터 크기에 따라 저장 방식을 동적으로 선택하는 하이브리드 접근법을 사용합니다.

```
#[derive(Debug, Clone)]
pub struct InlineOptions {
/// Maximum data size to inline.
pub max_data_inlined: u64,
/// Maximum outboard size to inline.
pub max_outboard_inlined: u64,
```

```
}

impl Default for InlineOptions {
fn  default() -> Self {
Self {
max_data_inlined: 1024 * 16, // 16KiB
max_outboard_inlined: 1024 * 16, // 16KiB
}
}
}
```

저장 전략 분류

1. 작은 블롭 (≤ 16KiB)

- **저장 위치**: 데이터베이스 내부 (인라인)
- **구현**: `DataLocation::Inline(Bytes)`
- **특징**:
- 메타데이터와 함께 데이터베이스에 직접 저장
- 빠른 액세스 속도
- 파일 시스템 오버헤드 없음

```
pub  enum DataLocation<T, U> {
Inline(T), // 작은 데이터: DB 내부 저장
Owned(U), // 큰 데이터: 별도 파일
```

```
External(Vec<PathBuf>, U), // 외부 파일 참조
}
```

2. 큰 블롭 (> 16KiB)

- **저장 위치**: 별도 파일 시스템
- **구현**: `DataLocation::Owned(size)`
- **파일 구조**:
- 데이터 파일: `{hash}.data`
- 아웃보드 파일: `{hash}.obao4`
- 크기 정보: `{hash}.sizes4`
- 비트필드: `{hash}.bitfield`

임계값 기반 선택 메커니즘

```
impl Options {
// 데이터 크기 기반 인라인 여부 결정
pub fn is_inlined_data(&self, data_size: u64) -> bool {
data_size <= self.inline.max_data_inlined
}

// 아웃보드 크기 기반 인라인 여부 결정
pub fn is_inlined_outboard(&self, outboard_size: u64) -> bool {
outboard_size <= self.inline.max_outboard_inlined
```

```
}

// 데이터와 아웃보드 모두 인라인 가능한지 확인
pub fn is_inlined_all(&self, data_size: u64) -> bool {
let outboard_size = raw_outboard_size(data_size);
self.is_inlined_data(data_size) && self.is_inlined_outboard(outboard_size)
}
}
```

부분 다운로드 지원

1. 상태 관리

```
pub enum BaoFileStorage {
Initial, // 초기 상태
Loading, // 로딩 중
Partial(PartialFileStorage), // 부분 다운로드
Complete(CompleteStorage), // 완전한 데이터
NonExisting, // 존재하지 않음
Poisoned, // 오류 상태
}
```

2. 점진적 완성 과정

- **비트필드 추적**: 다운로드된 청크 범위 추적

- **실시간 검증**: 수신 중 BLAKE3 해시 검증
- **상태 전환**: 부분 → 완전 상태로 자동 전환

장점

1. **확장성**

- 메모리 제약 없이 대용량 데이터 처리
- 디스크 용량에 따른 저장 한계
- 효율적인 메모리 사용

2. **영속성**

- 프로세스 재시작 후에도 데이터 유지
- 안정적인 데이터 보존
- 백업 및 복구 가능

3. **하이브리드 최적화**

- 작은 파일: 빠른 데이터베이스 액세스
- 큰 파일: 효율적인 파일 시스템 활용
- 메모리와 디스크의 장점 결합

4. **고급 기능**

- 부분 다운로드 완전 지원
- 복잡한 상태 관리
- 가비지 컬렉션 및 정리 기능

단점

1. **복잡성**

- 복잡한 상태 관리 로직

- 다중 액터 아키텍처
- 디버깅 및 유지보수 어려움

2. **성능 오버헤드**

- 디스크 I/O로 인한 지연
- 데이터베이스 트랜잭션 오버헤드
- 파일 시스템 동기화 비용

3. **리소스 요구사항**

- 디스크 공간 필요
- 파일 시스템 권한 필요
- 더 많은 시스템 리소스 사용

ReadonlyMemStore 분석

구현 특징

```
pub  struct ReadonlyMemStore {
client: ApiClient,
}
```

```
struct Actor {
commands: tokio::sync::mpsc::Receiver<proto::Command>,
tasks: JoinSet<()>,
idle_waiters: Vec<irpc::channel::oneshot::Sender<()>>,
data: HashMap<Hash, CompleteStorage>, // 읽기 전용 데이터
```

}

핵심 특징

1. **생성 시점 데이터 고정**

- 생성자에서 제공된 데이터만 서비스
- 런타임 중 데이터 추가/수정 불가능

2. **단순한 구조**

- 복잡한 상태 관리 없음
- 완전한 데이터만 저장
- 최소한의 기능 구현

3. **제한된 기능**

- 읽기 작업만 지원
- 태그 시스템 미지원
- 임포트/익스포트 제한

장점

1. **단순성**

- 구현이 매우 단순
- 디버깅 용이
- 오류 가능성 최소화

2. **성능**

- 메모리 기반 빠른 액세스
- 상태 변경 오버헤드 없음

- 동시성 처리 단순

3. **안정성**

- 데이터 변경 불가로 인한 높은 안정성
- 예측 가능한 동작

단점

1. **기능 제한**

- 읽기 전용으로 활용도 제한
- 동적 데이터 처리 불가능
- 고급 기능 미지원

2. **유연성 부족**

- 런타임 데이터 변경 불가능
- 확장성 제한

성능 비교

읽기 성능

1. **ReadonlyMemStore** > **MemStore** > **FsStore**
2. 메모리 기반 저장소가 디스크 기반보다 빠름
3. 상태 관리 복잡도가 성능에 영향

쓰기 성능

1. **MemStore** > **FsStore** (작은 파일) > **FsStore** (큰 파일)
2. 인라인 저장이 별도 파일보다 빠름

3. 디스크 동기화 오버헤드 고려 필요

메모리 사용량

1. **ReadonlyMemStore** < **MemStore** < **FsStore**
2. FsStore는 메타데이터만 메모리에 유지
3. 하이브리드 접근법으로 메모리 효율성 확보

사용 사례별 권장사항

MemStore 권장 사례

- **개발 및 테스트 환경**
- **소규모 데이터 처리** (< 1GB)
- **임시 데이터 캐싱**
- **높은 성능이 필요한 단기 작업**

FsStore 권장 사례

- **프로덕션 환경**
- **대용량 데이터 처리** (> 1GB)
- **영구 데이터 저장**
- **다중 사용자 환경**
- **백업 및 복구가 필요한 경우**

ReadonlyMemStore 권장 사례

- **정적 콘텐츠 서비스**
- **CDN 캐시 레이어**

- **테스트 데이터 제공**
- **단순한 읽기 전용 서비스**

구현 세부사항

임계값 설정 최적화

```
// 기본 설정 (16KiB)
impl Default for InlineOptions {
fn  default() -> Self {
Self {
max_data_inlined: 1024 * 16,
max_outboard_inlined: 1024 * 16,
}
}
}

// 사용자 정의 설정 예시
let  custom_options = InlineOptions {
max_data_inlined: 1024 * 64, // 64KiB로 증가
max_outboard_inlined: 1024 * 32, // 32KiB로 증가
};
```

하이브리드 저장 결정 로직

1. **데이터 크기 측정**
2. **아웃보드 크기 계산**
3. **임계값과 비교**
4. **저장 방식 결정**
5. **적절한 저장소에 데이터 저장**

상태 전환 메커니즘

```
// 부분 → 완전 상태 전환
if update.new_state().complete {
let data = std::mem::take(&mut partial.data);
let outboard = std::mem::take(&mut partial.outboard);
let data: Bytes = <Vec<u8>>::try_from(data).unwrap().into();
let outboard: Bytes = <Vec<u8>>::try_from(outboard).unwrap().into();
*state = CompleteStorage::new(data, outboard).into();
}
```

결론

iroh-blob의 저장소 구현체들은 각각 명확한 목적과 최적화 전략을 가지고 있습니다:

- **MemStore**: 최고 성능이 필요한 소규모 임시 데이터
- **FsStore**: 확장성과 영속성이 필요한 프로덕션 환경
- **ReadonlyMemStore**: 단순한 정적 데이터 서비스

하이브리드 접근법을 통해 FsStore는 작은 파일의 빠른 액세스와 큰 파일의 효율적인 저장을 동시에 제공하며, 16KiB 임계값을 통해 최적의 성능과 메모리 사용량의 균형을 달성합니다.